

PCO : labo 3 Rapport

Autheurs: Santiago Sugrañes et Anthony Pfister

1. Vue d'ensemble de l'implémentation

1.1 Gestion de la concurrence

Chaque classe utilise un mutex global pour protéger ses données partagées :

- `PcoMutex Mutex_hospital`
- `PcoMutex Mutex_clinic`
- `PcoMutex Mutex_ambulance`
- `PcoMutex Mutex_ins`
- `PcoMutex Mutex_supplier`

2. Implémentation détaillée par classe

2.1 Hospital

Sections critiques

`transferSickPatientsToClinic():`

```
Mutex_hospital.lock();
int nbPatientsToTransfer = stocks[ItemType::SickPatient];
Mutex_hospital.unlock();

// Appel externe sans mutex (évite deadlock)
int accepted = clinic->transfer(ItemType::SickPatient,
nbPatientsToTransfer);

if (accepted > 0) {
    Mutex_hospital.lock();
    stocks[ItemType::SickPatient] -= accepted;
    Mutex_hospital.unlock();
}
```

Choix de conception:

- Le mutex est libéré avant l'appel à `clinic->transfer()` pour éviter les deadlocks
- La mise à jour du stock se fait sous mutex après confirmation de l'acceptation

updateRehab():

```
Mutex_hospital.lock();
// Ici, tout se fait sous mutex car toutes les opérations sont critiques
for (auto& batch : rehabBatches) {
    batch.second--;
    if (batch.second <= 0) {
        batchesToFree.push_back(batch.first);
    }
}
// Mettre à jour les stocks et supprimer les batches
[...]
Mutex_hospital.unlock();

// Facturation externe (hors mutex, pour éviter deadlock)
for (int freedCount : batchesToFree) {
    insurance->invoice(...);
}
```

Choix de conception:

- Toutes les opérations sur **rehabBatches** et **stocks** sont protégées par le mutex
- La facturation est effectuée hors du mutex pour éviter les deadlocks

transfer():

```
Mutex_hospital.lock();
if (money <= 0) { // Hospital en dette
    Mutex_hospital.unlock();
    return 0;
}
int availableBeds = maxBeds - currentPatients;
Mutex_hospital.unlock();

// Calcul de l'acceptation sans mutex
if (availableBeds <= 0) return 0;

// Mise à jour du stock sous mutex
Mutex_hospital.lock();
stocks[...] += accepted;
rehabBatches.push_back({accepted, 5}); // Pour les rehab patients
Mutex_hospital.unlock();
```

Choix de conception:

- Vérification de la dette et calcul des lits disponibles sous mutex
- Libération du mutex pour les calculs intermédiaires
- Réacquisition pour la mise à jour finale

2.2 Clinic

Sections critiques

payBills() Gestion des deadlocks:

```
Mutex_clinic.lock();
for(auto it = unpaidBills.begin(); it != unpaidBills.end(); ){
    if(money >= it->second){
        int billAmount = it->second;
        Supplier* supplier = it->first;
        money -= billAmount;

        Mutex_clinic.unlock(); // Libération avant appel externe

        it = unpaidBills.erase(it);
        supplier->pay(billAmount); // Appelle Mutex_supplier

        Mutex_clinic.lock(); // Réacquisition
    } else {
        ++it;
    }
}
Mutex_clinic.unlock();
```

Choix de conception critique:

- **Libération du mutex avant l'appel externe:** `supplier->pay()` acquiert `Mutex_supplier`, donc on doit libérer `Mutex_clinic` pour éviter un deadlock
- **Erase après unlock:** L'itérateur peut être modifié après libération du mutex car on ne l'utilise plus
- **Réacquisition:** Le mutex est réacquis avant de continuer la boucle

orderResources():

```
Mutex_clinic.lock();
for (const auto& item : resourcesNeeded) {
    if (stocks[item] != 0) continue;
    Mutex_clinic.unlock();

    Supplier *supplier = chooseRandomSupplier(item);
    int bill = supplier->buy(item, 1); // Appel externe

    Mutex_clinic.lock();
    if (bill > 0) {
        stocks[item] += 1;
        unpaidBills.push_back({supplier, bill});
    }
}
Mutex_clinic.unlock();
```

Choix de conception:

- Le mutex est libéré pour chaque commande individuelle pour permettre la concurrence
- La mise à jour du stock se fait uniquement si l'achat a réussi (`bill > 0`)

treatOne():

```
Mutex_clinic.lock();
if (money < salary) {
    Mutex_clinic.unlock();
    return;
}
money -= salary;
nbEmployeesPaid++;

// Consommation des ressources
for (const auto& item : resourcesNeeded) {
    stocks[item]--;
}
stocks[ItemType::SickPatient]--;
stocks[ItemType::RehabPatient]++;
Mutex_clinic.unlock();
```

Choix de conception:

- Toutes les opérations sont atomiques sous le mutex
- Vérification des fonds avant traitement

processNextPatient():

```
Mutex_clinic.lock();
bool hasUnpaidBills = !unpaidBills.empty();
Mutex_clinic.unlock();

if (!hasUnpaidBills) {
    orderResources(); // Pas de mutex nécessaire (géré en interne)
}

Mutex_clinic.lock();
bool canTreat = hasResourcesForTreatment() && money >=
getEmployeeSalary(EmployeeType::TreatmentSpecialist);
Mutex_clinic.unlock();

if(canTreat){
    treatOne(); // Pas de mutex nécessaire (géré en interne)
}
```

Choix de conception:

- Lecture des conditions sous mutex, actions hors mutex
- Les méthodes appelées (`orderResources()`, `treatOne()`) gèrent leur propre mutex

2.3 Insurance

Sections critiques

`payBills()` Pattern identique à Clinich:

```
mutex_ins.lock();
for(auto it = unpaidBills.begin(); it != unpaidBills.end(); ){
    if(money >= it->second){
        int billAmount = it->second;
        Seller* beneficiary = it->first; // Sauvegarde avant erase
        money -= billAmount;
        it = unpaidBills.erase(it);
        mutex_ins.unlock();

        beneficiary->pay(billAmount); // Appel externe (peut acquérir
mutex)

        mutex_ins.lock();
    } else {
        ++it;
    }
}
mutex_ins.unlock();
```

Choix de conception critique:

- **Sauvegarde du pointeur avant erase:** L'itérateur devient invalide après `erase()`, donc on sauvegarde `beneficiary` avant
- **Libération avant appel externe:** Pour éviter les deadlocks avec `recipient->pay()`
- **Païement FIFO:** Les factures sont payées dans l'ordre d'arrivée

2.4 Ambulance

Sections critiques

sendPatients():

```
Mutex_ambulance.lock();
if (hospitals.empty() || money < salary || stocks[ItemType::SickPatient] ==
0) {
    Mutex_ambulance.unlock();
    return;
}

int nbPatientsToTransfer = 1 + rand() % 5;
nbPatientsToTransfer = min(nbPatientsToTransfer,
stocks[ItemType::SickPatient]);
Mutex_ambulance.unlock();

// Appel externe sans mutex
auto* hospital = chooseRandomSeller(hospitals);
int accepted = hospital->transfer(ItemType::SickPatient,
nbPatientsToTransfer);

if (accepted > 0) {
    Mutex_ambulance.lock();
    stocks[ItemType::SickPatient] -= accepted;
    money -= getEmployeeSalary(EmployeeType::EmergencyStaff);
    nbEmployeesPaid++;
    Mutex_ambulance.unlock();

    // Facturation externe (hors mutex)
    insurance->invoice(getCostPerService(ServiceType::Transport) *
accepted, this);
}
```

Choix de conception:

- Vérifications initiales sous mutex
- Appel externe (`hospital->transfer()`) sans mutex
- Mise à jour des stocks et paiement du salaire sous mutex
- Facturation hors mutex

2.5 Supplier

Fonctionnalités principales

Sections critiques

pay()

```
Mutex_supplier.lock();  
money += bill;  
Mutex_supplier.unlock();
```

Choix de conception:

- Opération simple mais nécessite un mutex pour la sécurité thread-safe
- Protège contre les accès concurrents lors des paiements multiples

buy():

```
auto it_stock = stocks.find(it);  
if (it_stock == stocks.end() || it_stock->second < qty) return 0;  
stocks[it] -= qty;  
return getCostPerUnit(it) * qty;
```

Note: Cette méthode n'utilise pas de mutex car elle est appelée depuis

`Clinic::orderResources()` qui gère déjà la synchronisation au niveau de la clinique.

3. Vérifications et tests

3.1 Tests unitaires

31 tests passés (ceux du template fourni, aucun test n'a été ajouté) couvrant:

Hospital (6 tests)

1. **ReceivesSickPatientsRespectingFundsAndBeds**: Acceptation de patients malades selon les lits disponibles et rejet si l'hôpital est en dette.
2. **ReceivesRehabPatientsStartsTimers**: Acceptation de patients en réhabilitation avec démarrage des timers (batchs de 5 jours) et facturation après libération.
3. **payNursingStaffDeductsMoneyAndCountsEmployees**: Paiement du personnel soignant avec déduction des fonds et comptage des employés.
4. **TransferSickPatientsToClinicMovesSomeAndInvoicesInsurance**: Transfert de patients malades vers la clinique avec facturation de l'assurance.

5. **HospitalPay** (test de concurrence): Thread-safety de la méthode `pay()` sous accès concurrents.
6. **EndToEndShortRunDoesMeaningfulWork**: Test end-to-end avec `DayClock` vérifiant le flux complet et la cohérence financière.

Ambulance (7 tests)

7. **SendPatients_Success**: Envoi réussi de patients vers les hôpitaux avec réduction du stock et modification des fonds.
8. **SendPatients_Fails_When_NoPatients**: Échec de l'envoi lorsqu'il n'y a pas de patients disponibles, sans changement d'état.
9. **SendPatients_Fails_When_NotEnoughMoneyForSalary**: Échec de l'envoi si les fonds sont insuffisants pour payer le salaire.
10. **PaySuccess**: Paiement simple qui augmente les fonds correctement.
11. **FundsDeltaMatchesSalaryAndTransportBill_OnSuccess**: Vérification du delta financier - `salary + transferred * transport_price` et facturation correcte.
12. **AmbulanceThrows**: Vérification que `buy()`, `transfer()`, `invoice()` lèvent `std::logic_error` et que `pay()` fonctionne.
13. **AmbulancePay** (test de concurrence): Thread-safety de la méthode `pay()` sous accès concurrents.

Clinic (9 tests)

14. **OrderResources_BuysMissing_OnNoStock**: Commande automatique de ressources manquantes (stock = 0) avec réduction des stocks fournisseurs et création de factures impayées.
15. **HasResourcesThenTreatOne_ConsumesResourcesAndMovesPatientToRehab**: Traitement d'un patient avec consommation de ressources, conversion malade → réhabilitation et paiement du salaire.
16. **TreatOne_Fails_WithoutMoney**: Échec du traitement si les fonds sont insuffisants, sans changement d'état.
17. **SendPatientsToRehab_InvoicesInsuranceAndReducesRehabStock**: Envoi de patients en réhabilitation vers les hôpitaux avec facturation de l'assurance.
18. **Transfer_AcceptsSickPatientsOnly_AndOnlyIfNoDebtAndNoUnpaidBills**: Acceptation uniquement de patients malades, avec rejet si dettes ou factures impayées.
19. **PayBills_PaysWhenFundsSufficient**: Paiement des factures aux fournisseurs lorsque les fonds sont suffisants.
20. **ClinicUnsupported**: Vérification que `buy()`, `invoice()` lèvent `std::logic_error` et que `pay()` fonctionne.
21. **ClinicPay** (test de concurrence): Thread-safety de la méthode `pay()` sous accès concurrents.

22. **Run_EndToEnd_TreatsAndTransfersAtLeastOnce**: Test end-to-end avec **DayClock** vérifiant le flux complet et la cohérence financière.

Insurance (3 tests)

23. **InvoiceQueuesBills_AndPayBillsPaysWhenFundsSufficient**: File d'attente des factures (FIFO) avec paiement uniquement lorsque les fonds sont suffisants.
24. **InsuranceThreading** (test de concurrence): Accumulation concurrente de factures sans perte et paiement correct de toutes les factures.
25. **InsuranceRun**: Test avec **DayClock** et méthode **run()** vérifiant le paiement en boucle avec fonds insuffisants puis suffisants.

Supplier (6 tests)

26. **SellsResource_MatchesCatalog**: Vérification que le catalogue correspond aux ressources vendues via **sellsResource()**.
27. **Buy_ReturnsZero_WhenInsufficientStock**: Retour de 0 (facture) si le stock est insuffisant, sans modification du stock.
28. **Buy_ReducesStock_AndReturnsCorrectBill**: Réduction du stock et calcul correct de la facture ($qty * cost_per_unit$).
29. **Pay_IncreasesFunds**: Paiement simple qui augmente les fonds correctement.
30. **PayIsThreadSafe** (test de concurrence): Thread-safety de la méthode **pay()** sous accès concurrents.
31. **ProducesItemsWhenFundsSufficient**: Test avec **DayClock** et méthode **run()** vérifiant la production de ressources et le paiement du personnel.

3.2 Tests de concurrence

Les tests vérifient également:

- **Thread-safety**: Pas de race conditions
- **Deadlock-free**: Pas de blocages lors d'appels concurrents
- **Cohérence des données**: Les invariants sont préservés

4. Décisions techniques importantes

4.1 Mutex globaux vs mutex par instance

Choix: Mutex globaux statiques

```
PcoMutex Mutex_hospital; // Global, un pour toute la classe
```

Justification:

- Simplicité d'implémentation
- Suffisant pour les besoins de ce labo

4.2 Facturation hors mutex

Choix: Tous les appels à `insurance->invoice()` sont effectués hors mutex

Justification:

- `invoice()` acquiert `mutex_ins`
- Évite les deadlocks potentiels
- L'ordre des factures est préservé par la file d'attente

5. Difficultés rencontrées et solutions

5.1 Deadlocks dans `payBills()`

Problème: Deadlock entre `Mutex_clinic` et `Mutex_supplier`

Solution: Libération du mutex avant l'appel à `supplier->pay()`

5.2 Itérateurs Invalides

Problème: Utilisation d'itérateurs après `erase()`

Solution: Sauvegarde des valeurs avant `erase()` et utilisation de la valeur retournée

5.3 Race condition sur les stocks

Problème: Accès concurrents aux stocks sans protection

Solution: Toutes les lectures/écritures de stocks sont protégées par mutex